



Esercitazione SQL

Python

Esercizio 1 – Gestione studenti

Crea una tabella chiamata Studenti con i seguenti campi:

- **id identificativo numerico univoco**
- **nome**
- **cognome**
- **eta**
- **corso**

Inserisci almeno 5 studenti.

Scrivi poi le query necessarie per:

1. **Visualizzare tutti gli studenti.**
2. **Visualizzare solo nome e cognome.**
3. **Visualizzare gli studenti con età maggiore di 20 anni.**
4. **Visualizzare gli studenti iscritti al corso "Informatica".**
5. **Ordinare gli studenti dal più giovane al più anziano.**



```
1. CREATE TABLE Studenti (  
2.   id INT PRIMARY KEY,  
3.   nome VARCHAR(50),  
4.   cognome VARCHAR(50),  
5.   eta INT,  
6.   corso VARCHAR(50)  
7. );
```

```
1. INSERT INTO Studenti VALUES  
2. (1, 'Mario', 'Rossi', 20, 'Informatica'),  
3. (2, 'Luca', 'Bianchi', 22, 'Economia'),  
4. (3, 'Anna', 'Verdi', 19, 'Informatica'),  
5. (4, 'Sara', 'Neri', 24, 'Matematica'),  
6. (5, 'Marco', 'Gialli', 21, 'Informatica');
```



```
1.-- 1
2.SELECT * FROM Studenti;
3.
4.-- 2
5.SELECT nome, cognome
6.FROM Studenti;
7.
8.-- 3
9.SELECT *
10.FROM Studenti
11.WHERE eta > 20;
12.
13.-- 4
14.SELECT *
15.FROM Studenti
16.WHERE corso = 'Informatica';
17.
18.-- 5
19.SELECT *
20.FROM Studenti
21.ORDER BY eta ASC;
```



Esercizio 2 – Gestione prodotti

Crea una tabella chiamata Prodotti con i seguenti campi:

- **id**
- **nome**
- **categoria**
- **prezzo**
- **quantita**
-

Inserisci almeno 6 prodotti appartenenti a categorie differenti.

Scrivi le query necessarie per:

1. **Visualizzare tutti i prodotti.**
2. **Visualizzare i prodotti con prezzo maggiore di 50 euro.**
3. **Visualizzare i prodotti con quantità minore di 10.**
4. **Visualizzare i prodotti appartenenti alla categoria "Informatica".**
5. **Modificare il prezzo di un prodotto.**
6. **Eliminare un prodotto tramite il suo id.**
7. **Ordinare i prodotti dal più costoso al meno costoso.**



```
1. CREATE TABLE Prodotti (  
2.   id INT PRIMARY KEY,  
3.   nome VARCHAR(50),  
4.   categoria VARCHAR(50),  
5.   prezzo DECIMAL(10,2),  
6.   quantita INT  
7. );
```

```
1. INSERT INTO Prodotti VALUES  
2. (1, 'Mouse', 'Informatica', 25, 20),  
3. (2, 'Monitor', 'Informatica', 200, 5),  
4. (3, 'Tastiera', 'Informatica', 60, 8),  
5. (4, 'Scrivania', 'Arredamento', 150, 4),  
6. (5, 'Quaderno', 'Cancelleria', 5, 30),  
7. (6, 'Cuffie', 'Audio', 80, 6);
```



```
1.-- 1
2.SELECT * FROM Prodotti;
3.
4.-- 2
5.SELECT *
6.FROM Prodotti
7.WHERE prezzo > 50;
8.
9.-- 3
10.SELECT *
11.FROM Prodotti
12.WHERE quantita < 10;
13.
14.-- 4
15.SELECT *
16.FROM Prodotti
17.WHERE categoria = 'Informatica';
18.
19.-- 5
20.UPDATE Prodotti
21.SET prezzo = 70
22.WHERE id = 3;
23.
24.-- 6
25.DELETE FROM Prodotti
26.WHERE id = 6;
27.
28.-- 7
29.SELECT *
30.FROM Prodotti
31.ORDER BY prezzo DESC;
```



Esercizio 3 – Gestione ordini e clienti

Crea un piccolo database per gestire gli ordini di un negozio.

Crea due tabelle.

Clienti

- **id_cliente** – chiave primaria
- **nome**
- **cognome**
- **citta**

Ordini

- **id_ordine** – chiave primaria
- **id_cliente** – chiave esterna collegata a Clienti
- **prodotto**
- **quantita**
- **prezzo**
- **data_ordine**

Inserisci almeno 5 clienti e 8 ordini, facendo in modo che alcuni clienti abbiano effettuato più ordini.

Scrivi le query necessarie per:

1. **Visualizzare tutti gli ordini.**
2. **Visualizzare gli ordini con prezzo superiore a 100 euro.**
3. **Visualizzare prodotto, quantità e prezzo ordinando i risultati dal prezzo più alto.**
4. **Visualizzare tutti i clienti che abitano in "Roma".**
5. **Utilizzare una JOIN per mostrare nome e cognome del cliente insieme al prodotto acquistato.**
6. **Calcolare il numero totale degli ordini.**
7. **Calcolare il prezzo medio degli ordini.**
8. **Calcolare quanto ha speso complessivamente ogni cliente utilizzando GROUP BY.**
9. **Mostrare solamente i clienti che hanno speso complessivamente più di 200 euro usando HAVING.**
10. **Mostrare il cliente che ha speso più di tutti.**



1. **CREATE TABLE Clienti (**
2. **id_cliente INT PRIMARY KEY,**
3. **nome VARCHAR(50),**
4. **cognome VARCHAR(50),**
5. **citta VARCHAR(50)**
6. **);**

1. **CREATE TABLE Ordini (**
2. **id_ordine INT PRIMARY KEY,**
3. **id_cliente INT,**
4. **prodotto VARCHAR(50),**
5. **quantita INT,**
6. **prezzo DECIMAL(10,2),**
7. **data_ordine DATE,**
- 8.
9. **FOREIGN KEY (id_cliente)**
10. **REFERENCES Clienti(id_cliente)**
11. **);**





```
1.-- 1
2.SELECT *
3.FROM Ordini;
4.
5.-- 2
6.SELECT *
7.FROM Ordini
8.WHERE prezzo > 100;
9.
10.-- 3
11.SELECT prodotto, quantita, prezzo
12.FROM Ordini
13.ORDER BY prezzo DESC;
14.
15.-- 4
16.SELECT *
17.FROM Clienti
18.WHERE citta = 'Roma';
19.
20.-- 5
21.SELECT
22.  Clienti.nome,
23.  Clienti.cognome,
24.  Ordini.prodotto
25.FROM Clienti
26.JOIN Ordini
27.ON Clienti.id_cliente = Ordini.id_cliente;
28.
29.-- 6
30.SELECT COUNT(*) AS numero_ordini
31.FROM Ordini;
32.
33.-- 7
34.SELECT AVG(prezzo) AS prezzo_medio
35.FROM Ordini;
36.
37.-- 8
38.SELECT
39.  Clienti.nome,
40.  Clienti.cognome,
41.  SUM(Ordini.prezzo) AS totale_speso
42.FROM Clienti
43.JOIN Ordini
44.ON Clienti.id_cliente = Ordini.id_cliente
45.GROUP BY
46.  Clienti.id_cliente,
47.  Clienti.nome,
48.  Clienti.cognome;
49.
50.-- 9
51.SELECT
52.  Clienti.nome,
53.  Clienti.cognome,
54.  SUM(Ordini.prezzo) AS totale_speso
55.FROM Clienti
56.JOIN Ordini
57.ON Clienti.id_cliente = Ordini.id_cliente
58.GROUP BY
59.  Clienti.id_cliente,
60.  Clienti.nome,
61.  Clienti.cognome
62.HAVING SUM(Ordini.prezzo) > 200;
63.
64.-- 10
65.SELECT
66.  Clienti.nome,
67.  Clienti.cognome,
68.  SUM(Ordini.prezzo) AS totale_speso
69.FROM Clienti
70.JOIN Ordini
71.ON Clienti.id_cliente = Ordini.id_cliente
72.GROUP BY
73.  Clienti.id_cliente,
74.  Clienti.nome,
75.  Clienti.cognome
76.ORDER BY totale_speso DESC
77.LIMIT 1;
```

Buon davante a tutti

